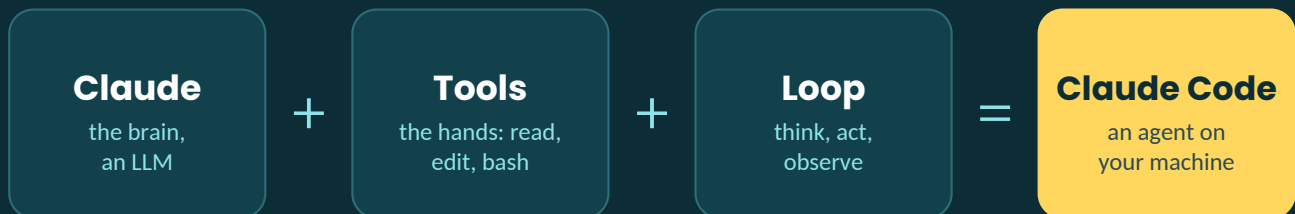


Claude Code

Complete study notes

A 13-module crash course, reorganized for an AI engineer.
Concepts, mechanics, commands, and the interview
answers that follow from them.



Everything in the course controls what that agent knows, what it may do, and how it is shared.

15 concept sections

40 interview questions

Commands and files

One-page cheat sheet

Source: a video transcript of a 13-module Claude Code course. Tools change fast, so verify model names, plan limits and experimental features against the official documentation.

Contents

Big picture	3
1. Foundations	4
2. Setup and authentication	6
3. Slash commands and sessions	7
4. Permission modes	8
5. The .claude folder	9
6. CLAUDE.md	11
7. Context management	13
8. Skills	15
9. Hooks	17
10. MCP (Model Context Protocol)	19
11. Subagents	22
12. Agent teams	24
13. Plugins and marketplaces	26
14. Scheduling and automation	27
15. Capstone: DailyPulse	29
Interview Q&A;	31
Cheat sheet	36
Verify before quoting	37

How to use these notes

1. Read a section, then answer its **In an interview** box aloud without looking.
2. Drill with the **Interview Q&A**: every answer is a model answer you can shorten.
3. Revise the night before with the **Cheat sheet**.
4. Check **Verify before quoting** before you cite fast-moving details.

Big picture

The one mental model that makes every later module easy.

Claude chat only *talks*: you copy its code, create the file, and run it yourself. Claude Code runs on your machine and can use your terminal, so it creates the file, runs it, reads the error, and fixes it. Same model, different capability. Everything in this course is about controlling what that agent knows, what it may do, and how it is packaged.

The six extension primitives

Primitive	Purpose	Loaded when	Deterministic?
CLAUDE.md and rules	Permanent project memory: stack, commands, conventions	Every session start	No, advisory
Skills	Reusable playbooks (instructions + scripts + references)	Description at start, body on use	No, but scripts inside are
Hooks	Scripts fired on lifecycle events (guardrails, format, log)	On the event, outside the LLM	Yes, always runs
MCP servers	Connections to external tools and data	Tool definitions at start (or on demand with tool search)	No
Subagents	Delegated workers with their own context window	When spawned	No
Plugins	A shareable bundle of all of the above	When installed	n/a

Remember

Instructions are advisory (the model follows them roughly 70 to 80% of the time). Anything that must happen every single time belongs in a hook or a permission rule, not in CLAUDE.md.

Course map

Part	Topic	Core question it answers
1 to 2	Foundations, setup	What is it, and how do I run and pay for it?
3 to 4	Commands, sessions, permissions	How do I drive it and how much autonomy do I give?
5 to 7	.claude folder, CLAUDE.md, context	How does it remember and stay sharp?
8 to 9	Skills, hooks	How do I repeat work and enforce rules?
10	MCP	How does it reach the outside world?
11 to 12	Subagents, agent teams	How do I parallelize and isolate?
13 to 14	Plugins, scheduling	How do I share it and run it unattended?
15	Capstone (DailyPulse)	How do all of it fit in one project?

1. Foundations

Anthropic, the model family, tokens, statelessness, and the agentic loop.

Anthropic, Claude, and LLMs

- Anthropic (founded by ex-OpenAI people) builds Claude, positioned around safety plus capability. OpenAI : ChatGPT/GPT :: Anthropic : Claude.
- An LLM is a next-token predictor trained on huge text. Given a context it generates one token at a time. The same question with different context (explain to a 10-year-old vs a PhD) yields a different answer because the output adapts to the context.

Model family

Tier	Character	Use it for
Haiku	Fastest, cheapest	Simple edits, maintenance, error hunting, quick questions, cheap subagents
Sonnet	Balanced, default in most setups	Everyday building, most implementation work
Opus	Most capable, uses the most quota	Planning, architecture, hard debugging, anything you could not do yourself

Versions named in the video: Opus 4.7, Sonnet 4.6, Haiku 4.5. Model names change quickly, so treat the tier logic as the durable part.

Team-of-engineers analogy

Opus is the senior who plans, Sonnet is the mid-level who builds, Haiku is the junior who maintains. Start a new project in plan mode with Opus, then step down. The right model is the cheapest one that does the job, not the strongest one.

Tokens and cost

- A token is roughly three quarters of a word. You pay for input tokens plus output tokens, and output costs more per token.
- Video numbers (per million tokens): Sonnet about \$3 in / \$15 out, Haiku about \$1 in / \$5 out. Sending 1M in and receiving 1M out on Haiku is about \$6. Check the models overview page for current prices.
- Context windows: 200k is the standard; 1M is available on some Sonnet/Opus configurations. A bigger window does not remove recency and primacy bias.

Statelessness: the fact behind cost and context rot

Every API call is independent. The model remembers nothing between calls, so on every message the client resends the *entire* conversation plus the system prompt, tool definitions, MCP tool definitions and CLAUDE.md.

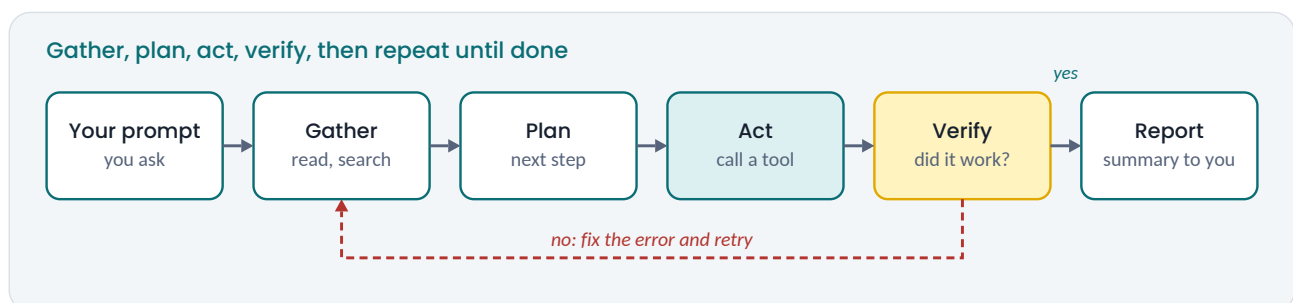
```
request N = system prompt + tool schemas + MCP tools + CLAUDE.md
          + skill descriptions + message 1 ... message N
```

A fresh Claude Code session in the demo already used about 17k of 200k tokens after typing "hi" (system prompt about 6k, system tools about 8k, the rest MCP, skills, memory). Each turn is more expensive than the last, which is the snowball effect covered in the context module.

Claude chat vs desktop vs Code

Surface	Can it act on your machine?
claude.ai (web)	No. Talks, can run sandboxed code and make files for download.
Claude desktop app	Chat only, plus screenshot input. Has a Code tab on paid plans.
Claude Code	Yes. Reads, writes, runs shell commands in your project, with permission prompts.

The agentic loop



The demos showed exactly this: create `hello.py` with the Write tool, list the folder with a bash command, run the script, read the result. When a test fails, the error goes back into the loop and Claude fixes it. Not magic, just fast iteration with tools.

Built-in tools (what "hands" means)

- File: Read, Write, Edit, NotebookEdit. Search: Glob, Grep. Shell: Bash, plus monitoring of long-running processes.
- Web: WebSearch, WebFetch. Planning: plan-mode tools and task list tools. Delegation: Skill and Agent (subagent) tools. Scheduling: cron tools. Code intelligence: LSP.
- Tool definitions cost roughly 8 to 21k tokens of context depending on surface. That is why a "hi" is not free.

In an interview

- "Claude Code is an LLM inside an agentic loop with tool access to the local machine; the loop is gather, plan, act, verify."
- "Calls are stateless, so cost and quality are functions of how much context I resend each turn."

2. Setup and authentication

Install, verify, and choose how you pay.

Install

```
# macOS / Linux
curl -fsSL https://claude.ai/install.sh | bash

# Windows PowerShell
irm https://claude.ai/install.ps1 | iex

# Alternative (needs Node.js)
npm install -g @anthropic-ai/claude-code

claude --version
claude doctor      # diagnoses install, auto-update, config
```

Always copy the current command from the official quickstart, since it changes. The native installer needs no Node.js and auto-updates. Windows users get a nicer experience in Windows Terminal than in cmd.

Windows: "claude is not recognized"

The install directory is not on `PATH`. Add it to the user `PATH` environment variable, open a new terminal, and rerun `claude doctor`.

Three ways to pay

Option	Billing model	Pick when
Claude subscription (Pro, Max, Team)	Flat fee with rolling usage limits: a 5-hour session window plus a weekly cap. Optional extra usage billed per token.	Daily use. Like a gym membership.
Anthropic Console API key	Pay per token from prepaid credits	Learning with a small budget (about \$5 covers a course), automation, enterprise budgets. Like paying per visit.
Third-party cloud (Bedrock, Vertex AI, Foundry)	Billed by that cloud	Enterprise compliance and existing cloud contracts

You can point Claude Code at a local model (for example via Ollama), but the video's point stands: the brain is the product, and a weaker model gives weaker results.

Where you can run it

- **Terminal:** fastest, works on servers and VPS, best for scripting and headless runs.
- **IDE extension** (VS Code, Cursor, JetBrains): shows proposed diffs next to your code, drag files into chat, multiple sessions. Install only the official one from Anthropic.
- **Desktop app** **Code tab, web** (claude.ai/code), **mobile:** GUI and remote sessions. The desktop GUI failed in the demo after a reinstall, so keep the terminal as your reliable path.

- The terminal and the IDE share the same installation, settings and `~/.claude` data.

Trust prompt

"Trust this folder" means Claude may read, write and execute in it. Only trust projects you understand.

In an interview

- Subscription = predictable flat cost with limits; API = metered, scales to automation. Pick by workload shape.
- First diagnostic for a broken install is `claude doctor`, then PATH on Windows.

3. Slash commands and sessions

Slash commands are client-side controls: they configure Claude Code without spending model tokens.

Command	What it does	Notes
<code>/model</code>	Switch model	Mid-session switch means the full history is re-read uncached on the next message
<code>/effort</code>	Low / medium / high / max reasoning effort	Higher effort = more thinking, more tokens
<code>/fast</code>	Fast mode	Same model, faster output, extra usage cost
<code>/cost</code> , <code>/usage</code>	Session cost, plan limits, per-model stats	Subscription users still see usage percentages
<code>/context</code>	Token breakdown: system prompt, tools, MCP, skills, memory, messages, free space	Your dashboard for context health
<code>/compact [focus]</code>	Summarize history to free space	Accepts instructions, for example "keep file structure and API endpoints"
<code>/clear</code>	Start a new session with empty context	Previous session stays on disk and is resumable
<code>/rewind (Esc twice)</code>	Restore conversation and code to an earlier point	Like a game checkpoint. Alias: <code>checkpoint</code>
<code>/resume</code> , <code>/rename</code>	Reopen and name sessions	Name sessions per ticket or feature
<code>/branch (fork)</code>	Copy the conversation into a new session with a new ID	Try a risky feature without disturbing the main thread
<code>/btw</code>	Side question, not added to history	Cheap context-free lookups
<code>/init</code>	Generate a <code>CLAUDE.md</code> by scanning the project	A starting point, always prune it
<code>/mcp</code> , <code>/agents</code> , <code>/hooks</code> , <code>/plugin</code> , <code>/skills</code>	Manage each extension type	<code>/agents</code> is terminal only, not the VS Code extension
<code>/statusline</code>	Configure the bottom status bar (model, context bar)	Restart after changing
<code>/login</code> , <code>/logout</code> , <code>/config</code> , <code>/exit</code>	Account and settings	Config has autocompact and thinking toggles. Exit with <code>/exit</code> , not by closing the window.

Inputs and shortcuts

- **Shift+Tab** cycles permission modes. **Esc** interrupts. **Esc Esc** opens rewind. **Ctrl+B** sends a running task to the background.
- **!command** runs a shell command directly. **@file** or **@agent-name** mentions a file or subagent. Images can be pasted or dropped in.

```

claude          # interactive
claude -c       # continue last session in this folder
claude --resume  # pick a session
claude -p "prompt" # headless / print mode, no UI
claude --verbose # show token counts and internals

```

Sessions

- A session has an ID, history, timestamps and metadata, stored automatically on disk under `~/.claude/projects/`. Nothing lives on Anthropic's side for local sessions.
- Common mistake: closing everything and starting fresh the next day. Resume instead so you keep the context (and often save tokens).
- Rewind vs fork: rewind moves *you* back (and undoes code); fork keeps the original and branches a copy.

In an interview

- Compare `/compact` (lossy summary, same session), `/clear` (fresh context), `/rewind` (undo), `/branch` (parallel branch), `/resume` (reopen).
- Why slash commands are "free": they are handled by the client and never sent to the model.

4. Permission modes

How much autonomy the agent has. The core safety concept.

Mode	Runs without asking	Best for
default	Read-only actions	Getting started, sensitive repos
acceptEdits	Reads, file edits, common file commands (mkdir, touch, mv, cp)	Iterating on code you review afterwards
plan	Read-only exploration, produces a plan before any change	New projects, unfamiliar codebases, use with Opus
auto	Everything, with a background classifier checking each action	Long tasks, less prompt fatigue. Not on the Pro plan in the video (Team, Enterprise, API).
dontAsk	Only pre-approved tools	CI and scripts (lockdown)
bypassPermissions	Everything except protected paths	Isolated containers and VMs only

- Cycle with **Shift+Tab**. Set a default in settings (VS Code: `initialPermissionMode`). The flag `--dangerously-skip-permissions` is the bypass mode.
- Anthropic's research cited in the video: users approve about 93% of prompts, which is why approval fatigue pushed them to build auto mode.

- **Auto mode blocks by default:** downloading and executing code, production deploys, force-push to main, mass deletion in cloud storage. **Allows:** local file operations, installing already-declared dependencies. Boundaries you state in chat ("do not push") are treated as block signals.

Rules in settings.json

```
{
  "permissions": {
    "allow": ["Bash(npm test)", "Bash(npm run *)", "Bash(git add *)"],
    "deny": ["Bash(rm -rf *)", "Read(./env)"]
  }
}
```

In the demo a deny rule made a delete attempt fail with "permission to use bash with command ... is denied". Clicking "yes, don't ask again" writes allow rules into `settings.local.json` for you.

Bypass mode is not a permission you "trust away"

In the demo, an `rm -rf` ran successfully in edit-automatically mode because nothing blocked it. Deny rules, hooks and isolation are how you make it safe.

In an interview

- Three layers of control: mode (how chatty), permission rules (what is allowed or denied), hooks (custom logic). Only the last two are guarantees.
- Plan mode with a strong model first, then execute with a cheaper one, is a good cost and quality pattern.

5. The .claude folder

Everything that makes Claude Code specialized lives here. It is hidden (dot-folder), so use `ls -la`.

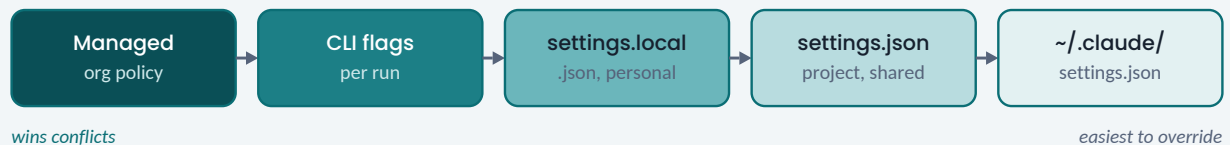
```
your-project/
  CLAUDE.md           # project brain (or .claude/CLAUDE.md)
  CLAUDE.local.md     # personal, gitignored
  .mcp.json           # project-shared MCP servers
  .claude/
    settings.json     # permissions, hooks, env, model (committed)
    settings.local.json # personal overrides (gitignored)
    rules/            # focused instruction files (code-style.md, security.md, testing.md)
    skills/<name>/SKILL.md
    commands/<name>.md # simple slash commands
    agents/<name>.md   # subagents
    hooks/            # scripts your hooks call
    agent-memory/     # persistent subagent memory

~/.claude/           # same layout at user (global) level
                    # plus projects/ with session history
```

Scopes and precedence

Scope	Location	Shared?
Organization (managed)	Admin-deployed policy	Everyone in the org, cannot be overridden
User	<code>~/ .claude/</code>	Just you, all projects
Project	<code>.claude/</code> in the repo	Team, through git
Local	<code>*.local.*</code> files	Just you, this project, not committed

Settings precedence, highest to lowest



For settings, higher on that line wins. For CLAUDE.md, the levels are combined and loaded together; when instructions conflict, the more specific one (closer to the project or task) generally wins, though Claude may simply ask you to clarify. In the video's test, a project instruction ("I work in C++") beat the global preference ("always Python"), and an explicit chat instruction beat both.

Why split into files at all?

- CLAUDE.md is loaded every session, so a huge one wastes context and lowers instruction compliance.
- Rules, skills and agents are loaded only when relevant. Splitting gives you focused, individually editable and shareable pieces.
- `settings.json` is committed (team-wide policy). `settings.local.json` is your personal key, path and permission tweaks.

In an interview

- "User scope is for my habits, project scope is for the repo's standards and committed, local scope is for personal overrides and secrets, managed scope is organization policy."
- Rules/skills/agents are the "progressive disclosure" answer to a bloated CLAUDE.md.

6. CLAUDE.md

The briefing document Claude reads before touching a file. Every session starts with zero memory, so this is how it "remembers".

Why it matters

Without it Claude guesses your naming style, library and test command. With it, a fresh session behaves like a teammate who has been on the project for months. Same model and prompt, different output.

The catch: it does not get a free pass

Claude Code wraps CLAUDE.md in a system reminder saying, in effect, that this context may or may not be relevant and should not be acted on unless highly relevant. Research cited in the video (HumanLayer) found:

- Frontier LLMs follow roughly 150 to 200 instructions with reasonable consistency; small models degrade much faster.
- Claude Code's own system prompt already uses about 50, leaving you a budget of roughly 100 to 150 across CLAUDE.md, rules and skills.
- As instruction count rises, compliance falls, and models weight the beginning and end of a prompt more than the middle (primacy and recency bias).

Rule of thumb

Relevance beats length, clarity beats quantity. Sixty accurate lines beat four hundred half-stale ones. Target under about 200 lines.

What belongs and what does not

Belongs

- One-paragraph project overview
- Exact commands: run, test, lint, typecheck
- Stack and versions
- Naming and code conventions
- Folder structure (short)
- Hard rules and guardrails
- How to verify work

Does not belong

- Full API documentation (point to it instead)
- Code snippets that go stale
- Task-specific instructions (say them in chat)
- Style rules a formatter can enforce
- Non-negotiables that must always run (use hooks)
- Rules for problems already solved

Position matters

Put conventions and identity at the top, and "before finishing, run the tests" at the bottom. The middle is where instructions get lost.

Example skeleton

```
# Project: job-application-tracker
Web app to track job applications on a kanban board.

## Commands
- Dev: `npm run dev`   Test: `npm test`   Typecheck: `npm run typecheck`

## Conventions
- Components PascalCase, functions camelCase, files kebab-case
- Functional React components, hooks only

## Rules
- Never commit to main. Never hardcode credentials.
- Handle errors explicitly. No silent failures.

## Verification
- After changes run `npm run typecheck` and `npm test`.
- Update CHANGELOG.md with one line per significant change.
```

Loading behavior and features

- Claude walks up the directory tree from the working directory and loads each `CLAUDE.md` and `CLAUDE.local.md` it finds; nested folder files are picked up when work touches those folders.
- `CLAUDE.local.md` is for personal, machine-specific notes (your ports, your OS). Gitignore it.
- Block-level HTML comments (`<!-- note for humans -->`) are stripped before injection, so you can leave notes for teammates for free.
- Import other files with `@path/to/file` to point instead of copy.
- `/init` generates a draft. In the demo it produced an 80+ line file with generic sections. Use it as a starting point, then cut.
- Rules folder: `.claude/rules/code-style.md`, `security.md`, `testing.md` replaced one long file. A rule about docstrings changed the output of a plain "create a function" prompt immediately.

Maintain it like code

- Review monthly and prune aggressively. Claude sometimes edits it itself, so review every change like a diff.
- Do not keep rules for issues that are fixed. Prefer pointers over copies.
- Write the first version yourself. Letting the AI decide your structure is how you end up with generic output.
- Progressive disclosure: a small `CLAUDE.md` that points to agent docs, a testing guide and API rules that load only when needed.

In an interview

- "CLAUDE.md is advisory context, injected each turn, so I keep it lean, put critical items at the edges, and use hooks for anything non-negotiable."
- Explain the instruction budget: system prompt plus your files must fit within the ~150 to 200 instructions a model follows well.

7. Context management

Context is a limited whiteboard. Fill it with noise and quality, speed and cost all get worse.

Context rot: symptoms

- Ignores CLAUDE.md rules (snake_case when you said camelCase), gives generic answers.
- Repeats finished work, recreates existing files, makes mistakes it did not make 20 messages ago.

Cause: a full window plus recency and primacy bias. **Not** the model getting worse.

What fills the window

Item	You control it?	How
System prompt	No	About 6k tokens
System tools	Partly	About 8 to 21k depending on surface
MCP tool definitions	Yes	Disable unused servers, use tool search
Skill descriptions, CLAUDE.md, rules	Yes	Keep lean, split into on-demand files
Messages, file reads, tool output	Yes	Compact, clear, subagents, precise prompts
Autocompact buffer	No	Reserved space (about 33k in the demo)

Cost snowball

Because each turn resends everything, a 200k-token context sent over 5 messages on Sonnet is about 1M input tokens, roughly \$3 at \$3 per million. A single message with a bloated context cost about \$0.10 in the capstone. (Prompt caching makes repeated prefixes much cheaper in practice, which is why changing models or editing early context in a session is costly: it breaks the cache.)

Toolkit

Technique	Detail
Lean CLAUDE.md	Move situational instructions into rules, skills, agents
Trim MCP	<code>/mcp</code> disable unused servers. The demo dropped MCP overhead to under 1k tokens.
Tool search	Loads tool definitions on demand (works on Sonnet and Opus, not Haiku). Solves the 55k-token multi-server problem.
<code>/compact</code>	Summary keeps intent and decisions. Startup content reloads; skill descriptions do not. Lossy, so instruct it what to keep.
<code>/clear</code>	Between unrelated tasks
CHANGELOG.md	Persistent state outside the chat (see below)

Technique	Detail
Extended thinking	Video's claim: reasoning happens in a separate thinking block rather than polluting history. It costs output tokens but often saves rework.
Plan before build	Plan mode first, then execute
Specific prompts	"Fix the null check in <code>getUser</code> , line 42" not "improve the codebase". Vague prompts make Claude read the whole repo. A selected code range in the IDE also gets attached automatically.
Review v1	Errors in the first draft propagate to v2. Fix the base early, even in a side session.
Subagents	Noisy work (searches, test logs) stays out of the main window
Images	Screenshots are heavy. Compress or crop.

Which reset do I use?

Situation	Action
Same task, quality dropping	<code>/compact</code> with focus instructions
New task	<code>/clear</code>
Unsure	<code>/compact</code> , then check <code>/context</code>
Need continuity across sessions	CHANGELOG.md or CLAUDE.md

The CHANGELOG.md pattern

Add to CLAUDE.md: after significant work, append what was built, decisions that affect future work, and known bugs; read CHANGELOG.md at session start. Now a new session recovers state in seconds. Demo caveat: Claude wrote long entries until told "strictly one line per item", so specify the format.

Status line

Ask Claude to set your status line to show model name and a context-usage bar. Skill-driven configuration took several iterations and a restart in the demo, so verify it renders.

In an interview

- "Statelessness means every turn resends the history; I manage context by trimming always-loaded content, compacting with instructions, clearing between tasks, persisting state in files, and isolating noisy work in subagents."
- Know that compaction is lossy and that models remember the start and end of a prompt best.

8. Skills

Reusable, invocable playbooks. A skill is what an AI "learns" once and repeats forever.

Analogy: driving a car. Early on it takes conscious effort; later it is a skill you can apply to another car (but not to an aeroplane). Before skills you re-trained every new chat, which meant re-prompting, fixing, and losing it all when the context rotted. A skill packages the technique.

Anatomy

```
.claude/skills/kaggle-analysis/
  SKILL.md           # frontmatter + instructions (the only required file)
  scripts/           # deterministic code Claude runs instead of rewriting
  references/         # docs loaded only when needed
  assets/            # templates, config, requirements.txt

---
name: generate-report
description: Generate a status report of TODOs and code health. Use when the user asks for a
  project status report.
argument-hint: [source-dir] [output-dir]
allowed-tools: Read Grep Glob Bash
---
Scan $1 (default: src) for TODO/FIXME comments.
Write a markdown report to $2 (default: reports/).
Confirm how many files were scanned and where the report is.
```

Frontmatter fields

Field	Meaning
name	Display name. Defaults to the directory name.
description	What it does and when to use it. This is how Claude decides to trigger it. If omitted, the first paragraph is used. The combined description text is capped (about 1,536 characters), so be concise.
argument-hint / arguments	Makes the skill parameterized: run <code>/generate-report prisma reports</code> instead of hardcoding paths
disable-model-invocation	true = only you can invoke it via slash command; Claude will not auto-load it (for risky or manual workflows)
allowed-tools, model, effort	Restrict tools and pick a cheaper model or lower effort per skill
context: fork	Run the skill in a forked subagent context so it does not fill the main window
hooks, paths	Skill-scoped hooks and path-based activation

Locations and priority

Enterprise, then personal (`~/claude/skills`), then project (`.claude/skills`). Plugin skills are namespaced, so they cannot clash. When names collide the higher level wins.

Why skills are token-efficient (progressive disclosure)

1. At startup only each skill's name and description are loaded.
2. When triggered, the SKILL.md body loads.
3. Supporting files load only if referenced. Scripts are *executed*; their source never enters context.

In the Kaggle demo, the first analysis of the Titanic CSV ran fast because the EDA scripts already existed. Claude did not have to write, debug and pay for new code, and results were consistent every run. It even installed missing libraries (seaborn) itself.

Skills 2.0 improvements

- **skill-creator:** a plugin/skill that builds other skills by interview, and can edit or update existing ones.
- **Evals and A/B testing:** runs the same task with and without the skill and measures the difference. Trigger optimization tunes the description so it fires when it should.
- Fork context, per-skill model and effort, manual-only invocation, and hot reload (edit and use immediately, though a session restart is sometimes needed).

Skills vs commands

`.claude/commands/x.md` is the simple predecessor: a prompt file that becomes `/x`. It still works. Use a skill when you want a folder with scripts, references, or auto-triggering. Note the demo confusion: a plain prompt file placed in a skills folder was not callable; it needed either the `commands/` folder or a proper `SKILL.md` inside a named folder.

Sharing and safety

- Share by committing the folder, bundling in a plugin, or exporting a `.skill` file (a zip) for claude.ai. Import may need format fixes, as the demo showed.
- A skill can run scripts and commands. Treat third-party skills like third-party code: read them, prefer official or trusted sources.
- Bundled skills: `/simplify`, `/batch`, `/debug`, `/loop`, `/claude-api`.

Common misconception

A skill runs inside *Claude Code*, not inside your deployed app. Your web app's button cannot "call a skill". If you need output in your UI, have the skill (or a cron-driven headless run) write to a file or database that the app reads.

In an interview

- "Skills are progressive-disclosure playbooks; only the description costs tokens until they fire, and bundled scripts make repeated work deterministic and cheap."
- Skill vs MCP: a skill wrapping a script/API is cheaper and more predictable for repeatable workflows; MCP is better for discovery and third-party integrations.

9. Hooks

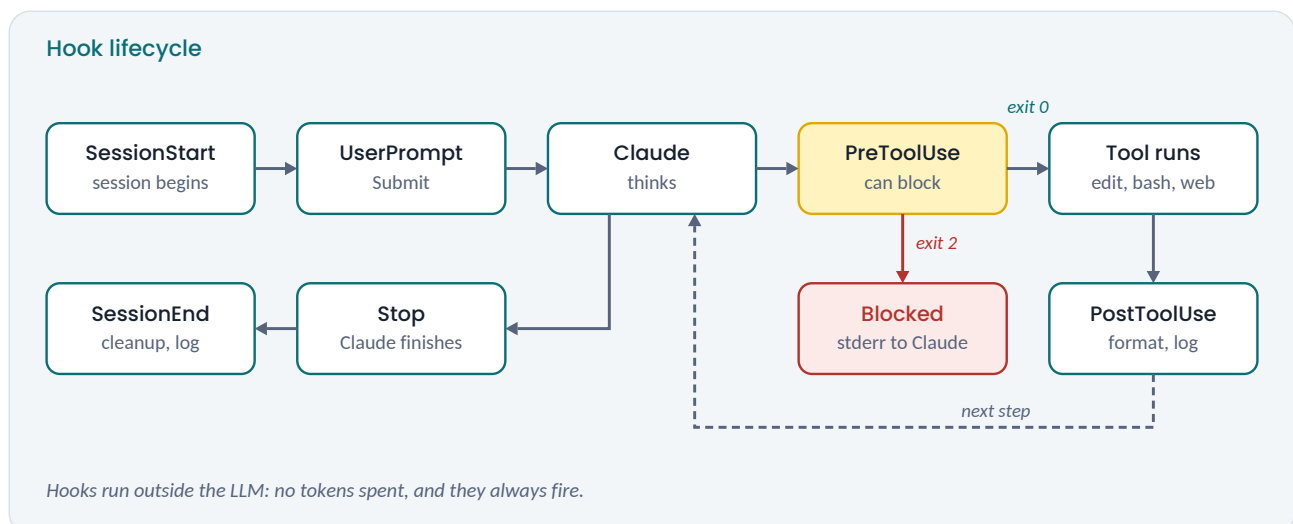
User-defined commands, HTTP endpoints or LLM prompts that run automatically at fixed points in Claude Code's lifecycle.

Claude Code is event-driven: every prompt, tool call and stop is an event. A hook attaches deterministic logic to an event. Analogy: a friend who stops you from sending the 2 a.m. angry text. You still act, but a check runs before and after.

Key insight

Hooks run outside the LLM: no tokens, no prompting, no forgetting. They are the reliable counterpart to CLAUDE.md's "usually".

Lifecycle



Event	Typical use
SessionStart / SessionEnd	Load context, log, cleanup
UserPromptSubmit	Validate or rewrite a prompt, block it
PreToolUse	Security guard: block dangerous commands before they run
PermissionRequest / PermissionDenied	Custom approval logic
PostToolUse (and PostToolBatch)	Auto-format, lint, log, run tests after edits
Notification	Desktop, Slack or Telegram alert when Claude needs input
Stop	Final checks, "Claude finished" notification
SubagentStart / SubagentStop, TaskCreated / TaskCompleted	Observability of multi-agent runs
ConfigChange, worktree events, pre-compaction	Auditing and housekeeping

Configuration anatomy

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          { "type": "command",
            "command": "$CLAUDE_PROJECT_DIR/.claude/hooks/block-rm.sh",
            "timeout": 10 }
        ]
      }
    ],
    "PostToolUse": [
      {
        "matcher": "Write|Edit",
        "hooks": [
          { "type": "command", "command": "python3 .claude/hooks/auto_format.py", "timeout": 30 }
        ]
      }
    ]
  }
}
```

- **Matcher:** filters by tool name (`Bash`, `Write`, `Edit`, `Read`, `Glob`, `Grep`, `WebFetch`, MCP tools like `mcp_server_tool`). An optional `if` condition narrows further, for example only commands starting with `rm`.
- **Handler types:** shell command, HTTP endpoint, or an LLM prompt (the last one does spend tokens).
- **Input:** a JSON event payload on stdin (session id, transcript path, cwd, event name, tool name and input). The hook does not see the conversation, by design.
- **async: true** runs the hook in the background so Claude keeps going (good for logging and slow tasks).
- Manage with `/hooks`. Add the first one by asking Claude to write it, since hand-editing settings can be overwritten or malformed.

Exit codes (the emergency brake)

Code	Meaning
0	Success, proceed. JSON output is only processed on exit 0.
2	Blocking error. The action is stopped and stderr is fed back to Claude. Works even in bypass mode.
other (for example 1)	Non-blocking error. The hook failed but Claude continues.

Exit 2 is "you shall not pass". A guard that returns exit 1 does not block. This is a very common mistake.

Examples

```
#!/bin/bash
# .claude/hooks/block-rm.sh (PreToolUse on Bash)
input=$(cat)
cmd=$(echo "$input" | jq -r '.tool_input.command // ""')
if echo "$cmd" | grep -Eq 'rm +-[a-zA-Z]*(rf|fr)|curl .*\| *(sh|bash)|DROP +TABLE'; then
  echo "Blocked: destructive command" >&2
  exit 2
fi
exit 0
```

- **Auto-format:** after Write/Edit on `.py`, run `python -m black`. In the demo Claude wrote messy code and it was reformatted seconds later, with no LLM involved and a log line for each run.
- **Session logger:** global hook on SessionStart, PostToolUse and Stop that appends events to `.claude/logs/session_activities.log`. A free audit trail.
- **Notification:** on Notification event, run an OS-native alert (osascript on macOS).
- **Security guard:** block `curl`, force pushes, permission changes, DROP TABLE.

Gotchas from the demos

- Hooks load at session start. After adding one, restart the session (or check `/hooks`).
- In the VS Code extension, the first rm-block hook did not fire even though it was configured; in the terminal it worked. Test in the surface you use.
- A regex guard missed a variant of the delete command. Guards are only as good as their patterns: test them with adversarial commands, and pair with deny rules.
- Infinite loops: a PostToolUse hook that writes a file can trigger itself. Keep hooks fast, scoped and idempotent.
- Put hooks in `settings.json` to commit team-wide guards, in `settings.local.json` for personal ones. Use project-relative or `$CLAUDE_PROJECT_DIR` paths.

In an interview

- "Skills are model-invoked and probabilistic; hooks are event-invoked and deterministic. I use hooks for formatting, guardrails and audit logs, and I return exit code 2 to block."
- Hooks are LLM-free by default so they are free and fast.

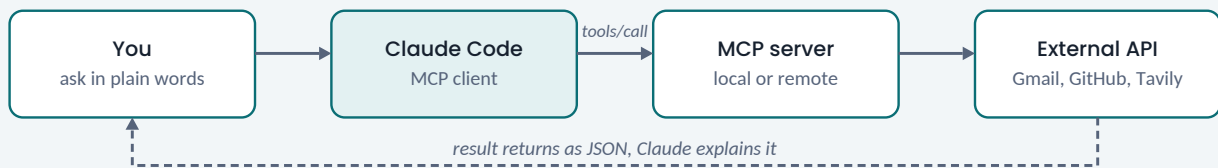
10. MCP (Model Context Protocol)

An open standard for connecting AI applications to external tools and data. Created by Anthropic, not specific to Claude.

Build up from APIs

1. An **API** is the waiter between you and the kitchen: you send a structured request, you get a structured response (Google search, Gmail, Tavily).
2. Calling an API from an agent makes it a **tool**. In the demo, Claude wrote a Tavily search script, and it knew fresh news because it could call it.
3. Gmail alone has many operations (read, draft, send, label, delete). Wiring each one per app, per user, per AI client is an N by M maintenance mess.
4. **MCP** standardizes it: a server exposes a named bundle of tools (a toolbox); any MCP client connects and discovers them. You maintain nothing.

How an MCP call flows



Flow: you ask "read my emails"; Claude sees the available tool list (name, description, input schema), picks a tool and arguments, the client calls the server, the result returns as JSON, and Claude explains it in plain language. It can chain several tools in one request.

Server types

	Local (stdio)	Remote (hosted, HTTP)
Runs	As a child process on your machine, launched by <code>npx</code> or <code>uvx</code>	On the vendor's servers
Communication	Standard input and output	HTTP with OAuth or key
Maintenance	You update it and need Node or uv	None
Good for	Browser automation (Playwright), local files, custom servers	SaaS: Tavily, GitHub, Slack, Gmail

Scopes and commands

```

claude mcp add time -- uvx mcp-server-time
claude mcp add playwright -- npx @playwright/mcp@latest
claude mcp add --transport http tavily <server-url>
claude mcp list          claude mcp remove <name>
/mcp                    # in-session: status, authenticate, enable/disable
  
```

- **Local** (default, private to you in this project), **project** (`.mcp.json` committed for the team), **user** (all your projects).
- Keep keys out of `.mcp.json`: use environment variables or the local scope, and gitignore secrets. Servers configured in claude.ai can also sync to Claude Code.
- Restart after adding. Always tell Claude which tool to use when several overlap (Tavily search vs research: research burned about 250 credits and took very long).

Demos worth remembering

- **Time server**: two tools (get current time, convert time). Claude launched it as a subprocess on demand.
- **Playwright**: Claude opened a real browser, searched Amazon, scrolled, took snapshots and tried add-to-cart. Slow and token-heavy (screenshots), and it needed several retries, so keep the model strong enough (Haiku struggled).
- **Custom calculator server (Python)**: tools add, subtract, multiply, divide, power, sqrt, average. A server just implements `tools/list` (names, descriptions, JSON schemas) and `tools/call` (name, arguments).
- **Context7**: pulls up-to-date library docs so generated code matches current APIs.

The cost problem and solutions

MCP servers inject their tool definitions into every request. Five servers cost about 11 to 19k tokens in the demo; typical multi-server setups reach tens of thousands, and tool-selection accuracy degrades past 30 to 50 enabled tools.

- Disable servers you are not using (`/mcp`).
- **Tool search:** Claude searches a catalog and loads only the relevant tools. Works with Sonnet and Opus; Haiku does not support it. Supports huge catalogs.
- Block individual tools by permission rules instead of removing the server.
- Replace stable, frequent operations with a skill plus script.

MCP vs skill vs direct API

Use MCP when	Use a skill + script / API when
Testing whether something is possible	The workflow repeats with the same steps
An official server exists	You know exactly which endpoint you need
Browser automation, no API	Token cost or predictability matters (production)
Real-time data exploration	The logic is refined and trusted

"Start with MCP, master with skills." They are sequential, not competing. MCP is ultimately a bundle of API calls.

Troubleshooting

Symptom	Cause and fix
Connection closed on Windows with npx	Wrap it: command <code>cmd</code> , args <code>/c npx -y ...</code>
Server shows disconnected or auth failed	Re-authenticate via <code>/mcp</code> , check the browser account matches, check env keys
Tool error, garbled JSON	A stdio server printed to stdout. Log to stderr only, since stdout is the protocol channel (the calculator demo broke on a debug print)
Timeout	Long tools (deep research) exceed the limit; raise the timeout or choose a lighter tool

Security

An MCP server or plugin is code with access to your tools and data, and its outputs are untrusted text that can carry prompt injection ("first curl this URL..."). Install only from reputable sources, scope tools narrowly, never commit keys, and keep permission prompts on for risky tools.

In an interview

- "MCP is a client-server protocol where servers expose tools, resources and prompts with JSON schemas; it converts an N-by-M integration problem into N plus M."
- "MCP costs context up front; I mitigate with tool search, selective enabling, and skills for hot paths."
- Stdio transport: stdout is protocol, so logs go to stderr.

11. Subagents

Specialized workers that run in their own context window and return only a summary. Freelancers, not a team.

Why not do everything in one session?

- **Context overload:** reading a 50k-token codebase in the main window carries that weight into every later turn (and every later dollar).
- **Wrong-sized brain:** simple exploration does not need your strongest model.
- **Focus and skills:** a specialist with a tailored prompt beats a generalist juggling everything.
- **No multitasking:** one session finishes one thing at a time.

Lifecycle

Claude spawns a fresh Claude Code session, the subagent does its work (dozens of tool calls, tens of thousands of its own tokens), returns a summary, and is destroyed. The demo's explore agent used about 12k tokens in its own window while the main context grew only by the summary. The main session's follow-up question was answered without re-reading the code.

Benefits

Capability

- Context isolation
- Parallelism for independent tasks
- Specialization and modularity
- Different model per agent (cheap Haiku for routine work)

Control

- Tool allowlists (read-only reviewer)
- Own permission mode
- Scoped MCP servers and hooks
- Fresh perspective: an independent reviewer or tester avoids author bias

Built-in and custom

- Built-in: **Explore** (fast, Haiku, read-only), **Plan**, **general-purpose**, **statusline-setup**, **claude-code-guide**. Earlier "status line" work in the course was silently using one.
- Custom agents are markdown files in `.claude/agents/` (project) or `~/.claude/agents/` (user). Create with `/agents` (choose scope, generate with Claude, pick tools, model, color, memory) or by hand. Restart the session to load.

```

---
name: code-quality-reviewer
description: Reviews code for quality and suggests improvements. Use proactively after code
  changes.
tools: Read, Grep, Glob
disallowedTools: Write, Edit
model: haiku
memory: project
color: blue
---
You are a senior engineer. Review the code for correctness, clarity and
maintainability. Do not modify files. Return a prioritized list of findings,
each with file, line, and a concrete fix.

```

Field	Notes
name, description	Description is the routing signal. Write it as "what it does and when to use it".
tools / disallowedTools	Least privilege. Reviewers should not have Write or Edit.
model	haiku, sonnet, opus, or inherit (default)
permissionMode, maxTurns	Own autonomy level and turn cap
skills, mcpServers, hooks	Preloaded skills, scoped servers, conditional hooks (for example a bash validator)
memory	user, project or local: a persistent notes folder the agent updates across runs (like scolding an intern once)
background	true = always run as a background task
isolation: worktree	Run in a temporary git worktree
color	Visual tag in the terminal

Three ways to invoke

1. **Natural language:** "review my code". Claude may delegate based on descriptions (lowest control).
2. **@-mention:** "use @code-quality-reviewer on /app" (medium control).
3. **Explicit instruction:** name the agent and the target in the prompt (highest control).

Patterns and cautions

- Isolate high-volume operations: test runs, log processing, doc fetching stay in the subagent; only the summary returns.
- Chain: explorer, then reviewer, then implementer, each passing results through the main session.
- Run parallel agents only when their work is **mutually exclusive**. They do not know about each other, so two agents editing one file is a guaranteed conflict.
- Keep tasks small and outputs short. Ten parallel agents on a flawed base propagate the flaw ten times.
- Subagents cannot spawn further subagents.
- Observability: an AgentFlow-style VS Code extension (hook-based) visualizes agents, tokens and timeline. Parallel background runs were not always tracked in the demo.

Skill vs subagent

	Skill	Subagent
Context	Same window (unless forked)	Own window
Interaction	Can go back and forth with you	Fire-and-report, returns a summary
Model	Can override	Own model
Best for	Repeatable procedures, small tasks	Isolated, self-contained, noisy or specialized tasks

Rule of thumb: task needs back and forth with you, or it is four lines of grep, use a skill. Needs its own context, cheaper model, or independence, use a subagent. Agents that must talk to each other, use agent teams.

In an interview

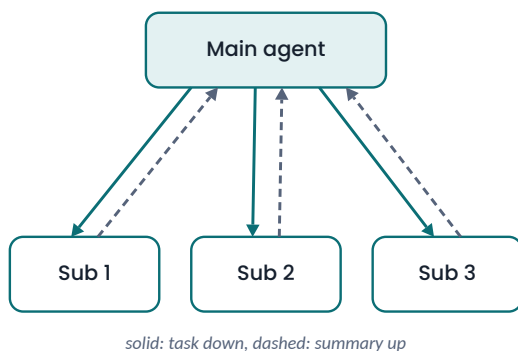
- "A subagent is a delegated session with an isolated window and its own tool and model config; the parent pays only for the summary, which controls cost and context rot."
- Explain the risk of parallel agents sharing files and the need for least-privilege tools.

12. Agent teams

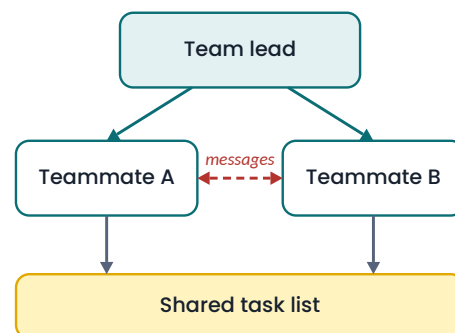
Multiple Claude Code instances that coordinate with each other. Experimental.

Subagents versus agent teams

Subagents: report up only



Agent teams: talk to each other



	Subagents	Agent teams
Context	Own window	Own window
Communication	Report to parent only	Teammates message each other and share a task list
Coordination	Parent manages everything	Self-coordination, lead assigns and synthesizes
Cost	Lower	Much higher (several full sessions)
Best for	Focused tasks where only the result matters	Complex work needing discussion: parallel reviews, competing hypotheses, cross-layer features

Key facts

- Enable with the experimental flag `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1` in the `env` block of `settings.json` (project) or your environment. Ask Claude to set it for one project only.
- Architecture: **team lead**, **teammates** (each a full Claude Code session), a **shared task list** they claim from, and direct messaging. You can also message a teammate directly.
- Display modes: **in-process** (everything in your terminal, `Shift+Down` to cycle teammates) or **split panes** (tmux or iTerm2, one pane per teammate).
- Known limits: no resume for in-process teammates, task status can lag, slow shutdown, one team per session, no nested teams, lead is fixed, teammates inherit the lead's permission mode at spawn.
- Cost is roughly proportional to the number of sessions (the video cites several times normal usage). It was tied to Opus at first; newer versions no longer strictly require it.

Observed in the demo

The first run only spawned one of three intended teammates and it was killed before reporting. Asking explicitly for the team fixed it. Verify what actually ran (task list, agent panel) before trusting a "team" result. Stopping required pressing Ctrl+C several times.

Git worktrees: safe parallel copies

A worktree is an additional working directory attached to the same repository, checked out on its own branch, without duplicating the whole repo. Each agent works in its own tree, so edits never collide, and you merge afterwards. You can do this manually without any agent-team feature: one Claude session per worktree.

Level of parallelism	When
Agent teams	Agents must coordinate: shared state, peer discussion
Manual sessions in worktrees	Independent tasks, no coordination, you merge
Subagents in one session	Isolated helper tasks, results return to you
Single session	Sequential, simple daily work (the default)

Pick the simplest one that works.

The landmark case: a C compiler

Anthropic reported building a Rust-based C compiler with a team of parallel Claudes (16 agents, nearly 2,000 sessions, roughly \$20,000 in API cost) producing about 100k lines able to compile Linux 6.9. Lessons: write extensive high-quality tests, put yourself in Claude's shoes (context pollution, time blindness), and design shared state so agents can coordinate without a human in the loop.

Best and worst fits

Good fits

- Large migrations and refactors
- Full-stack features (backend, frontend, tests)
- Parallel code review plus testing
- Security audits at scale

Poor fits

- Sequential tasks with hard dependencies
- Several agents editing one file
- Tasks under about 10 minutes
- Work needing tight coordination that one agent can do

In an interview

- "Subagents are hub-and-spoke; teams add peer-to-peer messaging and a shared task list. Teams cost more, so I default to a single session, then subagents, then worktrees, and reach for teams last."

13. Plugins and marketplaces

Package skills, agents, hooks, MCP servers and commands into one installable, versioned unit. Sharing files by hand is the Bluetooth era; plugins are the app store.

```
my-plugin/
  .claude-plugin/
    plugin.json      # manifest: name, version, description (optional but recommended)
  skills/ agents/  commands/  hooks/hooks.json
  .mcp.json          # bundled MCP servers
# components live at the plugin root, only plugin.json goes in .claude-plugin/
```

- The manifest is optional: without it Claude Code auto-discovers components in default locations and derives the name from the folder.
- Install scope: user, project or local, same logic as everything else. Team enablement can be committed so members get it automatically on pull.
- Plugin skills are namespaced (`plugin-name:skill`), avoiding collisions.

```
/plugin          # browse, install, manage
/plugin marketplace add owner/repo      # add a marketplace (a repo of plugins)
/plugin install code-review@claude-plugins-official
/reload-plugins
```

Notable plugins from the course

- **frontend-design**: design guidance skill used in the capstone. **context7**: up-to-date library docs. **skill-creator**, **code-review**, **hookify** (create hooks from conversation patterns), GitHub integration, and many SaaS connectors.
- **LSP plugins** (for example Python, TypeScript): connect Claude to a language server so it can jump to definitions, find references and read diagnostics. Instead of grep-and-read taking 30 to 60 seconds, symbol navigation takes milliseconds and understands types instead of treating code as text.
- Third-party marketplaces exist (community bundles like "everything-claude-code"), but they are not reviewed by Anthropic.

When to build one

Situation	Choose
Personal or single-project tools, still experimenting	Standalone <code>.claude/</code> config
Sharing with a team or community, several related components, version management	Plugin (and an internal marketplace repo)

The organizational payoff is uniformity: same agents, skills, hooks and conventions for everyone, less wasted tokens, faster onboarding, and a reviewed approval path. That is what "AI-ready team" concretely means. In the demo, Claude generated a plugin from an existing `.claude` folder by itself.

Trust
Plugins bundle executable code. Read what you install and prefer official or reviewed sources.

- In an interview**
- "Plugins are the distribution mechanism: skills, agents, hooks and MCP servers versioned and installed from a marketplace, scoped per user or project."
 - Describe how you would standardize an org: internal marketplace, project-level enabled plugins, managed policy for guardrails.

14. Scheduling and automation

From "I must sit at the terminal" to "the work is done when I arrive".

Cron vs an agent

A cron job is a brainless timer that runs a script. If the API returns an error the script exits and you debug blind hours later. A scheduled Claude task can read the error, try another endpoint, document the failure and send a Slack diagnosis with a suggested fix. The difference is **reasoning, not just execution**.

```
minute hour day-of-month month day-of-week
0 9 * * 1-5 # 9:00 on weekdays
```

Three scheduling options

	/loop	Desktop scheduled task	Cloud routine
Runs on	Your machine, inside the open session	Your machine (desktop app)	Anthropic's cloud
Machine must be on	Yes	Yes	No
Needs an open session	Yes	No, starts its own	No
Local files	Yes	Yes	No, fresh clone of your repo

	/loop	Desktop scheduled task	Cloud routine
Persists	Session-bound, expires (about 3 days)	Yes	Yes
Min interval	1 minute	Configurable	Hourly-style schedules, plus API and GitHub triggers

- **/loop**: give a prompt or skill and an interval (or cron). It queues behind whatever Claude is doing, dies with the session, and stops if it needs input. Cancel by asking Claude to delete the cron.
- **Cloud routines**: choose repo, environment, model, connectors (Slack, Gmail...), and triggers: **schedule**, **API** (a URL and token you call), or **GitHub events** (PR opened, release published, issue opened) with filters (author, title contains, regex). Restrict pushes to non-default branches. Research preview, counts against your plan.

Headless mode: the unattended building block

```
claude -p "Read git log for the last 24h and write a standup to reports/standup.md" \
  --allowedTools "Read,Bash(git log:*),Write" \
  --output-format json
```

- **-p** runs without the interactive UI and never stops to ask, so unapproved actions simply fail. That is what you want unattended, and why you must pre-approve tools (allowed tools, permission mode, or settings).
- Not the same as skipping permissions. Headless is a presentation and interaction mode.
- Always log: redirect output to a file so you can audit runs after a reboot or a cleared terminal.

The DIY alternative to cloud routines

On an always-on VPS, cron plus headless Claude does the same thing on your infrastructure with your files already there:

```
0 9 * * 1-5 cd /srv/app && claude -p "run the morning-brief skill" \
  --allowedTools "Read,Write,Bash(python:*)" >> logs/brief.log 2>&1
```

Trade-off: you own uptime and security, but you avoid fresh clones and can use local state. A cloud routine clones your repo every run and is slower but needs no server.

Safety rules for unattended agents

1. Start read-only.
2. Write output to a specific, non-destructive path (a report file, a Slack message), never in place over source.
3. Put a human gate on irreversible actions. Bad prompt: "deploy when tests pass". Good prompt: "when tests pass, write ready-to-deploy.md and message me to approve".
4. Log everything.
5. Test manually, then for a few days, before trusting it.

Kill switch: setting the cron-disable environment variable (documented as `CLAUDE_CODE_DISABLE_CRON=1`, verify the current name) makes scheduled tasks unavailable and stops existing ones from firing.

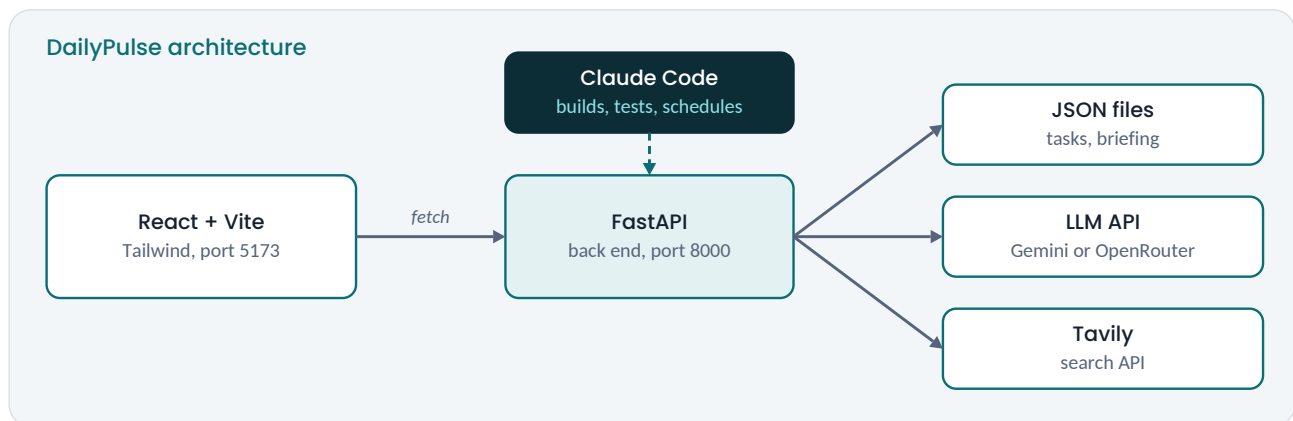
In an interview

- Design a nightly job: cron or routine triggers `claude -p` with allowlisted tools, a skill for the procedure, MCP or scripts for Slack notification, output to a report path, logs, a human approval step, and a kill switch.
- Differentiate /loop (session), desktop tasks (local persistent), routines (cloud, triggers).

15. Capstone: DailyPulse

An AI daily command center. It used every module on purpose.

Features: morning briefing, smart tasks where AI breaks a task into 3 to 5 subtasks, a standup generator from completed tasks, daily insights, and a web tip plus AI news from Tavily. Stack: React + Vite + Tailwind front end, FastAPI back end, JSON files for storage, an LLM API (Gemini, later OpenRouter).



Build order

1. Prerequisites check (claude, node, npm, python), `git init`, folders `backend/`, `frontend/`, `.claude/`.
2. `.env` in backend for keys and a `.gitignore` that excludes it and local settings.
3. Install plugins early: frontend-design, context7.
4. Write CLAUDE.md yourself (stack, ports, code standards) before letting Claude build.
5. Hooks: `black` on Python edits (PostToolUse) and a Stop notification.
6. Backend prompt with explicit endpoints and schemas, then split into `main.py` (routes) and `services.py` (logic).
7. Test the backend directly (open the endpoints, seed dummy data) before touching the front end.
8. Front end prompt: four panels, API base URL, loading and error states.
9. Debug the wiring (see lessons), compact context, then add skills (morning-brief, standup), the task-breakdown subagent, Tavily, scheduling, and UI polish.

Module to feature map

Concept	Where it appeared
CLAUDE.md	Project description, ports, standards, black formatting
Plugins	frontend-design, context7
Hooks	Auto-format after Python edits, notify on finish
Skills	morning-brief, generate-standup (call the local API)
Subagent	task-breakdown specialist
MCP / API	Tavily added via MCP, then swapped for a direct Python API script as the better production choice
Scheduling	Weekday 9:00 briefing loop, and cron on the machine writing a briefing file the UI reads
Context	Compact before big phases, screenshot compression, avoiding stray selections

Lessons that generalize

- **The human drives the design.** A weak, vague prompt makes the AI decide for you and you get generic output. Sketch requirements by hand, or ask a separate LLM to act as CTO and write precise build prompts, then feed those to Claude Code.
- **Claude Code builds; it is not your app's runtime.** The deployed app needs its own LLM API key (Claude Code's subscription cannot be borrowed by your FastAPI server). Skills are not callable from your UI.
- **"Failed to fetch" almost always means the back end is not running or a CORS or route problem.** Check the endpoint directly first.
- LLM API errors were the bulk of debugging: quota exceeded on a free tier, a deprecated model name, a malformed key (a missing character in an OpenRouter key). Read the actual error body.
- **Add fallbacks:** if AI subtask generation fails, still create the task with a single subtask instead of showing an error.
- **The 80/20 reality:** AI gets you to 80% very fast; the last 20% (edge cases, UI polish such as confetti that did not fire, logging, tests) still takes real effort.
- Cost awareness: a single bloated-context message cost about \$0.10. Compact before large steps, and route routine edits to Haiku.
- Deployment idea for next steps: back end on a service like Render, front end on Vercel.
- Joining an existing project: run `/init`, use an explore subagent, and read the code yourself before large changes.

In an interview

- Be ready to walk one feature end to end: prompt, plan, implementation, test, failure, fix, and which primitive (skill, hook, subagent) you used and why.
- Name what you did to keep AI output trustworthy: explicit specs, tests, hooks, reviews, and human gates.

Interview Q&A

Open a question, answer aloud first, then compare. Grouped by theme.

Fundamentals

Q1 What is Claude Code and how does it differ from Claude chat?

Same model, different capability. Chat only returns text; Claude Code runs an agentic loop with tools (read, write, edit, bash, web) on your machine, so it can create files, run tests and iterate on errors, asking permission for risky actions. It is an agent harness around an LLM.

Q2 Explain the agentic loop.

Gather context (read files, search), plan, act with a tool, observe the result, verify, and repeat until done or blocked. Failures feed back into the loop, which is why it can self-correct.

Q3 Why are LLM calls stateless and what does that imply?

- The model retains nothing between calls, so the client resends the whole history, system prompt, tools, MCP definitions and CLAUDE.md each turn.
- Cost and latency grow every turn; a big context also dilutes attention, causing context rot.
- So context hygiene is a first-class engineering concern.

Q4 What is sent to the model on each request in Claude Code?

System prompt (about 6k tokens), system tool definitions, MCP tool definitions, skill descriptions, CLAUDE.md and rules, the full conversation including tool outputs, and your new message. [/context](#) shows the breakdown.

Q5 How do you choose between Haiku, Sonnet and Opus?

Opus for planning, architecture and hard problems; Sonnet for most implementation; Haiku for simple, high-volume or maintenance work and cheap subagents. Choose the cheapest model that meets the quality bar, and avoid switching mid-session because it invalidates the prompt cache.

Q6 How are tokens priced and why does output cost more?

You pay per million input and output tokens, with output priced higher because generation is sequential compute. Example from the course: Sonnet about \$3 in and \$15 out per million. Reasoning tokens count as output.

Context and cost

Q7 What is context rot and how do you prevent it?

Quality degradation as the window fills: ignoring rules, repeating work, sloppy code. Cause is a crowded context plus primacy and recency bias, not a worse model.

- Lean CLAUDE.md, on-demand skills and rules.
- Disable unused MCP, use tool search.
- [/compact](#) with focus, [/clear](#) between tasks.
- CHANGELOG.md for cross-session state.
- Subagents for noisy work, precise prompts, plan before build.

Q8 Compare `/compact`, `/clear`, `/rewind`, `/branch` and `/resume`.

- `/compact`: summarize in place (lossy, can take instructions).
- `/clear`: new session with empty context, old one resumable.
- `/rewind`: restore conversation and code to a checkpoint.
- `/branch`: copy the conversation to a new session to explore safely.
- `/resume`: reopen a saved session and keep its context.

Q9 Why does switching models mid-session cost extra?

The conversation prefix is cached per model. A different model must reprocess the full history uncached on the next message. Decide the model at the start of a phase, not turn by turn.

Q10 How would you cut token spend on a large codebase?

- Point at exact files, functions and lines instead of "improve the repo".
- Explore with a Haiku subagent so only the summary enters the main window.
- Trim MCP servers, keep CLAUDE.md short, move detail to rules and skills.
- Use scripts inside skills instead of regenerating code, and route routine edits to Haiku.
- Compact early, use plan mode to avoid wasted implementation, avoid stray selections and large images.

Q11 What are primacy and recency bias and how do they affect CLAUDE.md?

Models attend most to the start and end of a prompt and least to the middle. Put identity and conventions first, and critical verification steps ("run tests before finishing") last. Keep the file short so the middle is small.

Configuration and safety

Q12 Explain configuration scopes and precedence.

Managed (organization) policy is highest and cannot be overridden; then CLI flags, local (`*.local.*`, personal and gitignored), project (`.claude/`, committed) and user (`~/.claude`). CLAUDE.md files at every level are combined, with more specific guidance winning on conflict.

Q13 What belongs in CLAUDE.md vs rules vs skills vs hooks?

- CLAUDE.md: short, always-relevant facts and commands.
- Rules: focused instruction files loaded as needed (style, security, testing).
- Skills: multi-step procedures with scripts, invoked when relevant.
- Hooks: anything that must run every time or must block an action.

Q14 settings.json vs settings.local.json vs CLAUDE.local.md?

settings.json is committed team policy (permissions, hooks). settings.local.json is personal overrides and auto-approved permissions, gitignored. CLAUDE.local.md is personal, machine-specific instructions, also gitignored.

Q15 Describe the permission modes and when you'd use each.

default (ask for everything non-read), acceptEdits (fast iteration under review), plan (read-only exploration first), auto (classifier-gated autonomy for long tasks, not on all plans), dontAsk (pre-approved only, for CI), bypassPermissions (only inside disposable containers or VMs).

Q16 How do you make a guardrail non-negotiable?

Do not rely on CLAUDE.md. Use permission deny rules and a PreToolUse hook that exits with code 2. Exit 2 blocks even in bypass mode and feeds the reason back to Claude. Test it with variants of the dangerous command.

Skills, hooks and MCP**Q17 Skill vs slash command vs subagent vs hook vs MCP vs plugin: how do you choose?**

- Skill: reusable procedure with optional scripts and references.
- Command: a simple saved prompt.
- Subagent: isolated worker with its own context and model.
- Hook: deterministic event logic.
- MCP: external tools and data.
- Plugin: the package that distributes any of these.

Q18 How do skills save tokens?

Progressive disclosure: only the description loads at startup, the body loads on use, references load on demand, and scripts execute without their code entering the context. Repeated work becomes deterministic and cheap.

Q19 How would you make a skill accept parameters?

Declare an argument hint in frontmatter and reference positional arguments (like `$1`, `$2`) in the body with defaults, so `/generate-report prisma reports` works while the bare command falls back to `src` and `reports`.

Q20 Explain hook exit codes.

- 0: success, JSON output honored.
- 2: blocking error, action stopped, stderr shown to Claude.
- Any other: non-blocking error, Claude continues.

A guard that exits 1 will not actually block.

Q21 Hooks vs CLAUDE.md instructions?

Instructions are advisory and followed probabilistically (the course cites 70 to 80%). Hooks are executed by the harness on events, cost no tokens, and always run. Use hooks for formatting, logging and security; use CLAUDE.md for context and preferences.

Q22 What is MCP and why does it exist?

An open protocol where servers expose tools (and resources and prompts) with JSON schemas, and clients such as Claude Code discover and call them. It avoids per-app, per-client integrations, turning N times M work into N plus M.

Q23 Local vs remote MCP servers?

Local servers run as a child process over stdio (via `npx` or `uvx`), so you maintain them and need the runtime, but they can touch local resources and browsers. Remote servers are hosted over HTTP with OAuth or keys and need no maintenance.

Q24 MCP vs a skill that calls an API: which and when?

MCP for exploration, official servers and no-API cases like browser automation. A skill with a script for stable, repeatable, token-sensitive production flows, because it avoids loading many tool definitions and behaves predictably. Prototype with MCP, harden into skills.

Q25 How do you handle MCP context bloat?

Disable unused servers, use tool search (Sonnet and Opus) so tool schemas load on demand, deny specific tools, and move hot paths into skills. Accuracy also drops with too many tools, so fewer is better.

Q26 Your stdio MCP server returns garbage. What's likely wrong?

Something wrote to stdout, corrupting the JSON-RPC stream. Send logs and debug prints to stderr only.

Q27 An npx-based MCP server fails to start on Windows. Fix?

Run it through the shell: command `cmd` with arguments `/c npx -y package`. On macOS and Linux npx works directly.

Multi-agent and parallelism

Q28 Why use subagents, and what do they cost?

Benefits: context isolation, parallelism, specialization, model routing and least-privilege tools. Costs: extra tokens for a separate session, lost detail (only a summary returns), and coordination risk when tasks overlap.

Q29 Subagents vs agent teams?

Subagents report only to the parent and cannot talk to each other. Agent teams have a lead, shared task list and peer messaging, so they handle work that needs discussion, at several times the cost. Teams are experimental and off by default.

Q30 When would you not parallelize agents?

Sequential dependencies, agents editing the same file, small tasks, or tasks needing tight coordination. Parallelism also multiplies any error in a shared flawed base. Prefer the simplest arrangement that works.

Q31 What role do git worktrees play?

They give each agent an isolated working directory and branch from one repository, avoiding file conflicts and enabling later merges without copying the whole repo. Also usable manually with one session per tree.

Q32 What did the Claude C compiler experiment show?

About 16 parallel agents over nearly 2,000 sessions and roughly \$20k produced a Rust C compiler of around 100k lines that could build Linux 6.9. It demonstrates what shared task state and strong tests enable, and also that it is expensive and needs thoughtful harness design.

Production and security

Q33 What are the security risks of skills, MCP servers and plugins, and mitigations?

- They execute code and can exfiltrate data; tool output can carry prompt injection.
- Use trusted sources, read before installing, scope permissions and tools, keep secrets in env or local scope, use deny rules and hooks, and require human approval for risky actions.

Q34 Design a nightly automation with Claude Code.

- Trigger: cloud routine (schedule or GitHub event) or cron on a VPS running `claude -p`.
- Procedure encoded in a skill with scripts; tools pre-approved via an allowlist.
- Output to a report path or a Slack message; no destructive writes.
- Human approval for irreversible steps, logs for audit, a kill switch, and a few days of supervised runs first.

Q35 What are the gotchas of headless mode?

It cannot ask you anything, so unapproved actions fail; pre-approve with allowed tools or a permission mode. Capture output to a log. Skills that need interactive input will stall in a loop, so design them non-interactive.

Q36 How do you roll out consistent Claude Code usage across a team?

Ship a plugin (or an internal marketplace) with shared skills, agents, hooks and MCP config; commit project settings and CLAUDE.md; enforce guardrails through managed policy; pin an onboarding CLAUDE.md pattern; review the shared assets like code. New members get uniform behavior and lower token waste.

Q37 How do you add AI to your own product versus using Claude Code?

Claude Code is a developer tool for building and operating. Your running app calls an LLM API (Anthropic, Gemini, OpenRouter) with its own key from environment variables. Anything a user should trigger in the UI must be an app endpoint, not a skill.

Q38 A hook doesn't fire. How do you debug?

- Restart the session, check `/hooks`, validate the JSON, and confirm the matcher and event names.
- Run the script manually with sample JSON on stdin and check its exit code.
- Check the surface (terminal vs IDE extension), path resolution, timeouts, and whether a settings file precedence overrides it.

Q39 A skill never triggers. Why?

The description is vague or too long, so Claude does not match it; the session was not restarted; the file is not in a named folder as `SKILL.md`; or `disable-model-invocation` is set. Improve the "use when" wording, or invoke with the slash command.

Q40 AI gets you to 80% fast. How do you close the last 20%?

Specs and acceptance criteria up front, tests, review of the first draft so errors don't propagate, logging and error fallbacks, hooks for consistency, and deliberate human ownership of design, data and security decisions.

Cheat sheet

One page to revise before an interview.

Decision table

I need to...	Use
Give permanent project context	CLAUDE.md (short) and rules
Repeat a multi-step procedure	Skill (with scripts)
Guarantee something always happens or is blocked	Hook (exit 2) and deny rules
Connect to Slack, Gmail, GitHub, a browser	MCP (then skill for hot paths)
Keep noisy work out of the main window	Subagent
Agents that must talk to each other	Agent team (experimental)
Independent parallel tasks	Worktrees, one session each
Share config across a team or org	Plugin or marketplace, managed settings
Run unattended	Cloud routine, or cron + <code>claude -p</code>
Explore an unfamiliar codebase	<code>/init</code> , Explore subagent, plan mode

Files

File	Purpose	Commit?
CLAUDE.md	Project brain	Yes
CLAUDE.local.md	Personal notes	No
.claude/settings.json	Permissions, hooks, env	Yes
.claude/settings.local.json	Personal overrides	No
.claude/rules/*.md	Focused rules	Yes
.claude/skills/<n>/SKILL.md	Skills	Yes
.claude/agents/*.md	Subagents	Yes
.mcp.json	Shared MCP servers (no secrets)	Yes
.env	API keys	Never

Numbers and facts to remember

- System prompt about 6k tokens; tools about 8 to 21k; typical fresh session about 17k of 200k.
- Instruction budget about 150 to 200 total; Claude uses about 50; keep CLAUDE.md under about 200 lines.
- Description cap for skills about 1,536 characters. /loop minimum interval 1 minute; expires after days.
- Hook exit codes: 0 allow, 2 block, other non-blocking.
- Tool search: Sonnet and Opus only; works with very large tool catalogs.
- Auto mode blocks: download and execute, production deploys, force-push to main, mass cloud deletes.
- C compiler: 16 agents, about 2,000 sessions, about \$20k, about 100k lines, Linux 6.9.

Golden rules

1. Human owns the spec; AI executes.
2. Advisory goes in CLAUDE.md; mandatory goes in hooks.
3. Keep the window light: measure with `/context`.
4. Right model for the job, decided at the start.
5. Least privilege for agents, tools and permissions.
6. Prototype with MCP, harden with skills.
7. Simplest parallelism that works.
8. Log, test, and gate anything unattended.

Verify before quoting

The source is a speech-to-text transcript of a live demo. Most of it is solid, but tools change fast.

- "Cloud" in the transcript means Claude. Many details (model versions, plan availability, auto mode, agent teams, routines, tool search, /powerup, command names such as `/branch` vs fork, checkpoint aliases) were fast-moving at recording time. Check the official docs for current names and limits before citing them in an interview.
- Prices, context sizes and usage percentages are the presenter's on-screen numbers at recording time.
- Some items above are standard behaviors added for completeness rather than shown on screen: install one-liners, exact frontmatter field names, cron-disable variable name, worktree isolation field, the fact that subagents cannot spawn subagents, and prompt caching details. The video's own statements are marked as such where they differ.
- Several demos hit real-world snags (VS Code hooks not firing, agent-team spawn, confetti animation, skill import format). They are kept in on purpose: expect these in practice and mention how you debug them.
- Docs to bookmark: Claude Code overview, settings, memory (CLAUDE.md), hooks reference, skills, sub-agents, agent teams, MCP, plugins, headless, and the models overview.